

Wolf Pack Survival Adventure Game

Data Structures Lab Final Project

Project Overview

Build an **Interactive Text-Based Wolf Survival Game** where players control a lone wolf navigating through a dangerous wilderness. The game features story-driven decisions, resource management, combat encounters, pack dynamics, and multiple endings. This project demonstrates how data structures power game mechanics, AI decision-making, and narrative branching.

Why This Project?

- Highly engaging and creative (story-driven game)
- Different from typical business applications
- Uses data structures in unique gaming contexts
- Decision trees create branching narratives
- Combines logic with creativity
- Fun to build and test!

Project Duration: 7 Days (One Week)

Team Structure (8-10 Members)

Role Distribution:

1. Team Leader (1 person)

- Coordinate all modules
- Oversee story integration
- Manage testing and balancing
- Final quality assurance

2. LaTeX Report Specialist (1 person)

- Document game design and implementation
- Create professional technical report
- Include game mechanics explanations and flowcharts
- **WARNING:** Report is 15% of grade!

3. Core Developers (6-8 people)

- **Module 1: Story & Decision System (2 people)** - Decision trees, narrative branching
 - **Module 2: Character & Inventory (2 people)** - Stats management, linked list inventory
 - **Module 3: Combat & Events (2 people)** - Priority queue for threats, event system
 - **Module 4: GUI & Game Loop (2 people)** - Interface, save/load system
-

Learning Objectives

By completing this project, you will:

- Implement binary trees for decision-based narratives
 - Use priority queues for threat management and event scheduling
 - Apply stacks for undo/game state history
 - Use linked lists for dynamic inventory systems
 - Implement queue for action sequences
 - Create game AI using data structures
 - Build interactive GUI for storytelling
 - Understand game state management
-

Game Story & Setting

Background:

You are a **young wolf** separated from your pack during a harsh winter storm. You must survive in the wilderness, make critical decisions, build alliances, and find a new pack or establish your own. Every choice matters - some lead to survival, others to doom.

Game Objectives:

- **Survive 30 days** in the wilderness
 - **Maintain stats:** Hunger, Health, Energy
 - **Make decisions** that affect story outcome
 - **Collect resources:** Food, herbs, territory knowledge
 - **Build pack:** Recruit wolves, establish hierarchy
 - **Face threats:** Bears, rival packs, hunters, harsh weather
 - **Multiple endings:** Alpha wolf, lone wanderer, tragic death, etc.
-

System Requirements

Core Features (MUST HAVE):

1. Story & Decision System

- Implement using **Binary Decision Tree**
- Each node represents a story scenario with choices
- Left child = Choice A, Right child = Choice B
- Leaf nodes = Story endings
- Support multiple branching paths based on decisions

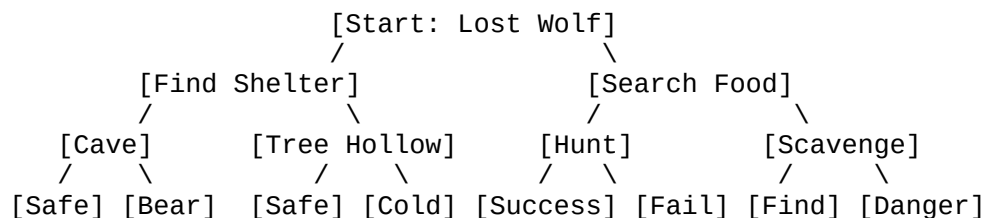
Example Decision Node:

Scenario: "You hear howling in the distance..."

Choice A: "Approach cautiously" → New scenario

Choice B: "Ignore and hunt" → Different scenario

Tree Structure:



Implementation Requirements:

- Minimum 20 decision nodes
- At least 5 different endings
- Track player path through tree
- Display current scenario and choices

2. Wolf Character & Stats Management

- **Health:** 0-100 (dies at 0)
- **Hunger:** 0-100 (dies if reaches 100)
- **Energy:** 0-100 (affects success rates)
- **Reputation:** 0-100 (affects pack interactions)

Stats Management:

- Hunger increases by 5 every decision/action
- Health decreases if hunger > 80
- Energy decreases with activities, restores with rest
- Stats affect available choices and success probabilities

Implement using: Simple struct/class with validation

3. Inventory System

- Store items: Food (meat, berries), Herbs (healing), Territory Maps
- Implement using **Singly Linked List** (dynamic size)
- Each item has: Name, Type, Effect, Quantity

Example Items:

- Rabbit Meat: Reduces hunger by 30
- Healing Herbs: Restores 20 health
- Territory Map: Reveals safe paths
- Wolf Scent: Helps recruit pack members

Operations:

- Add item to inventory
- Use item (remove from inventory, apply effect)
- Display all items
- Check if item exists
- Maximum 10 items (full inventory)

4. Threat & Event System

- Random events occur based on decisions and time
- Implement using **Priority Queue** (Min Heap)
- Events have priority levels:
 - Priority 1 (Critical): Bear attack, hunter nearby
 - Priority 2 (Urgent): Severe hunger, injury worsening
 - Priority 3 (Normal): Weather change, prey spotted

Event Processing:

- Higher priority events trigger first
- Events can chain (bear attack → injury → need healing)
- Some events are time-based (every 5 decisions)
- Display warning before critical events

5. Pack Management (Optional but Recommended)

- Recruit other wolves to form pack
- Each wolf has: Name, Role (Hunter/Scout/Guard), Loyalty
- Store pack members in **Linked List**
- Pack benefits:
 - Hunting success rate increases
 - Protection from threats
 - Shared resources
- Pack challenges:
 - Must feed all members
 - Loyalty decreases if hunger is high
 - Alpha challenges from strong wolves

6. Game State History

- Track previous game states for undo/back functionality
- Implement using **Stack**
- Store: Current scenario, stats, inventory, day number
- Operations:
 - Save state before each decision
 - Undo last decision (pop from stack, restore state)
 - Clear history
 - Maximum 5 undo levels

7. Action Queue

- Some actions take multiple turns (hunting, traveling)
- Implement using **Queue**
- Example: "Hunt deer" → Queue [Track → Chase → Kill → Eat]
- Process one action per turn from queue
- Display ongoing action progress

GUI Requirements (MANDATORY - Keep Simple):

Single Window Game Interface:

Top Section - Status Panel:

- Display wolf stats (Health, Hunger, Energy bars)
- Current day counter
- Weather/time indicator
- Pack size (if applicable)

Middle Section - Story Display:

- Large text area showing current scenario
- Narrative text with descriptions
- Scrollable for longer text

Choice Section - Decision Buttons:

- 2-4 choice buttons depending on scenario
- Clear button labels ("Hunt the deer", "Rest in cave")
- Disabled buttons for invalid choices (not enough energy, etc.)

Bottom Section - Game Controls:

- Inventory button (opens inventory panel)
- Stats details button
- Undo button (go back one decision)
- Save/Load game buttons
- Quit game button

Side Panel - Additional Info:

- Recent events/notifications
- Active effects (injured, well-fed, etc.)
- Quest objectives (optional)

GUI Library Options:

- **Dear ImGui** (Recommended - simple text and buttons)
 - **Qt Widgets** (Good for text display)
 - **SFML with text rendering** (If team wants graphics)
-

Technical Requirements

Programming:

- **Language:** C++ (C++11 or higher)
- **GUI:** Dear ImGui or Qt
- **Random Events:** Use C++ <random> library

Data Structures Required:

Feature	Data Structure	Why?
Story Decisions	Binary Tree	Branch based on choices
Threat Management	Min Heap Priority Queue	Critical events first
Inventory	Singly Linked List	Dynamic item storage
Pack Members	Singly Linked List	Variable pack size
Game History (Undo)	Stack	LIFO for backtracking
Action Sequences	Queue	FIFO for multi-turn actions

Game Logic:

```
// Pseudo-code for decision processing
void processDecision(Choice choice) {
    // Save current state to stack
    gameHistory.push(currentState);

    // Move to next node in decision tree
    currentNode = (choice == A) ? currentNode->left : currentNode->right;

    // Apply stat changes
    updateStats();

    // Check for triggered events
    checkEvents();

    // Generate random encounters based on probability
    if(random(0, 100) < encounterChance) {
        triggerEvent();
    }

    // Increment day
    dayCounter++;
}
```

File Management:

- Save game state to file
- Load game state from file
- Store: Current node ID, stats, inventory, pack, day
- Auto-save every 5 decisions

Validation:

- Cannot make choices without enough energy
 - Cannot use items not in inventory
 - Cannot recruit wolves with low reputation
 - Check if stats reach critical levels (death conditions)
-

AI Tools & Resources

Recommended AI Assistants:

1. Claude (Anthropic)

- Excellent for story writing and branching narrative design
- Great for binary tree implementation
- Good at game balance suggestions

2. ChatGPT (OpenAI)

- Good for GUI implementation
- Helpful for random event generation
- Debug game logic

3. GitHub Copilot (Optional)

- Speeds up repetitive game logic

Online Resources:

- **Binary Tree Tutorial:** <https://visualgo.net/en/bst>
 - **Game Design Patterns:** <https://gameprogrammingpatterns.com/>
 - **Random Number Generation:** <https://cplusplus.com/reference/random/>
 - **ImGui Examples:** <https://github.com/ocornut/imgui>
-

Prompt Engineering Guide

Bad Prompts:

```
"Make wolf game"  
"Decision tree code"  
"Game GUI"
```

Good Prompts:

For Binary Decision Tree:

```
"Create a Binary Tree in C++ for a story-based game. Each node  
has: scenarioID (int), description (string), choiceA_text (string),  
choiceB_text (string), and pointers to left and right children  
(next scenarios). Leaf nodes represent endings. Implement insert(),  
traverse(), and getCurrentChoices() functions. Show how to build  
a sample tree with 5 nodes."
```

For Priority Queue Events:

```
"Implement a Min Heap Priority Queue for game events. Each event  
has: eventName (string), priority (1-3, lower is more urgent),  
description (string), and effect on stats (struct). When player  
takes an action, check if events should trigger. extractMin()  
should return the highest priority event. Show insert and  
extractMin with example events like 'Bear Attack' (priority 1)  
and 'Found Berries' (priority 3)."
```

For Game State Stack:

"I need to implement undo functionality in my game using a Stack. Each state contains: currentNodeID (int), health (int), hunger (int), energy (int), inventoryList (linked list pointer), dayNumber (int). When player makes a decision, push current state. When player presses undo, pop and restore previous state. Implement push, pop, and restoreState functions. Handle empty stack case."

For Linked List Inventory:

"Create a Singly Linked List for game inventory. Each item node has: itemName (string), itemType (enum: FOOD, HERB, TOOL), effect (int), quantity (int). Implement addItem(), useItem(itemName), removeItem(), displayInventory(), and checkIfFull() functions. Maximum 10 items allowed. Show how using an item removes it from list and applies its effect to player stats."

For Story Writing:

"Help me write a branching story scenario for a wolf survival game. The wolf is lost and starving. Create one scenario node with:
1. Detailed description (3-4 sentences) setting the scene
2. Two meaningful choices that affect gameplay differently
3. Consequences for each choice (stat changes, new situations)
Make it immersive and realistic to wolf behavior. Include survival elements like hunger, danger, and territory."

Pro Tips:

- **Separate Story from Code:** Write story scenarios first, implement later
 - **Balance Testing:** Ask AI to suggest stat values for game balance
 - **Request Variations:** "Give me 3 different random events for this scenario"
 - **Debug with Context:** Share full game state when asking about bugs
-

Development Phases (7 Days)

Day 1: Story Design & Setup

- Brainstorm complete story outline (beginning, branches, endings)
- Create story flowchart on paper/whiteboard (20+ scenarios)
- Design decision tree structure
- Set up ImGui/Qt environment
- Define all structs (Wolf, Item, Event, GameState)
- Assign scenario writing to team members

Day 2-3: Core Systems Implementation

- **Module 1:** Implement binary tree for story
- **Module 1:** Write 20+ scenario texts and choices
- **Module 2:** Implement Wolf stats management
- **Module 2:** Implement inventory linked list
- **Module 3:** Implement priority queue for events
- **Module 3:** Implement game history stack
- **Module 3:** Implement action queue
- Test each structure independently

Day 4-5: Integration & Game Loop

- **Module 4:** Build complete GUI interface
- **Module 4:** Create game loop (display → choose → update → repeat)
- **All Modules:** Connect systems:
 - Decision button → Move in tree + update stats
 - Use item button → Apply effect + remove from inventory
 - Undo button → Pop stack + restore state
 - Random event trigger → Priority queue insertion
- Implement save/load functionality
- Add sound effects or ASCII art (optional)

Day 6: Content & Balancing

- Write more scenarios and endings (reach 20+ nodes)
- Balance game difficulty:
 - Adjust stat changes
 - Tune event probabilities
 - Test survival rate
- Add pack recruitment mechanics
- Playtest with team (each person plays through)
- Fix bugs and improve user experience
- Add tutorial/help screen

Day 7: Documentation & Demo

- Complete LaTeX report with:
 - Story flowchart
 - Data structure diagrams
 - Game mechanics explanation
 - Take screenshots of different scenarios
 - Record demo video showing:
 - Multiple playthroughs
 - Different choices and endings
 - Inventory and stats in action
 - Undo functionality
 - Save/load
 - Final polish and submission
-

Evaluation Criteria (30 Points)

Component	Points
Data Structures	5
Game Functionality	5
GUI & UX	5
Story Quality	5
LaTeX Report	5
Demo Video	5

Bonus Points (up to +5):

- ASCII art or simple graphics: +2
 - Multiple save slots: +1
 - Achievement system: +1
 - Sound effects: +1
-

Deliverables

1. Source Code

- All .cpp and .h files
- Story content files (scenarios.txt)
- Build instructions
- Sample save files

2. LaTeX Report (10-15 pages)

Must Include:

- Title page
- Game Design Document:
 - Story overview
 - Game mechanics
 - Win/lose conditions
- System Architecture:
 - Decision tree structure (diagram)
 - Priority queue usage for events
 - Stack-based undo system
- Data Structure Implementation:
 - Binary tree code and explanation
 - Priority queue for threat management
 - Inventory linked list
- Story Flowchart (visual representation)
- Testing & Game Balance:
 - Playtest results
 - Stat balance decisions
 - Bug fixes
- AI Tools Usage
- Conclusion

3. Demo Video (8-10 minutes)

Must Show:

- Start new game (introduction)
- Make 5-7 decisions showing branching
- Use inventory items
- Encounter random event
- Use undo button
- Reach one ending (win or lose)
- Show pack recruitment (if implemented)
- Demonstrate save/load
- Show different playthrough briefly

4. Game Design Document (Separate or in Report)

- Complete story map with all nodes
 - All possible endings
 - Item list with effects
 - Event list with triggers
-

Sample Story Scenarios

Scenario 1: The Beginning

"You wake up disoriented. The storm has passed, but you're alone. Your pack is nowhere to be seen. Snow covers everything, and your stomach growls with hunger. In the distance, you smell something - fresh blood. But approaching means leaving the area where your pack might return."

Choice A: "Follow the scent of blood (Hunt)"

- Leads to hunting scenario
- Energy -10, might find food

Choice B: "Wait here and howl for your pack"

- Leads to waiting scenario
- Energy -5, might encounter rival pack

Scenario 2: The Hunt

"You track the blood scent to a frozen stream. A wounded deer stands on the other side, weak and vulnerable. Easy prey. But the ice looks thin - it might not support your weight. You could also try to find another way around."

Choice A: "Cross the ice carefully"

- 70% success rate if energy > 50
- Success: Get deer (-50 hunger)
- Fail: Fall through ice (-30 health, -20 energy)

Choice B: "Find another way around"

- Takes 2 turns (queue: Move → Move → Hunt)
- Deer might escape (50% chance)

Ending Examples:

- **Alpha Wolf:** Form pack, defend territory, thrive
 - **Lone Wanderer:** Survive alone, never find pack
 - **Tragic Death:** Starve, freeze, or killed by hunters
 - **Reunion:** Find original pack after journey
 - **Sacrifice:** Save other wolves, die heroically
-

Challenge Tasks (Bonus Points)

Choose ANY for bonus:

1. ASCII Art Graphics

- Draw wolf, scenery, threats in ASCII
- Display in GUI alongside text
- Different art for different scenarios

2. Multiple Save Slots

- Allow 3 save game slots
- Display save date/time and progress
- Load any saved game

3. Achievement System

- Track accomplishments (survive 30 days, recruit 5 wolves)
- Store achievements in file
- Display in GUI

4. Sound Effects

- Howling, growling, wind sounds
 - Play on specific events
 - Use simple audio library
-

Important Warnings

Academic Integrity:

- AI helps with story and code structure
- You must understand the game logic
- Copying complete games = ZERO
- Story writing shows creativity

Common Pitfalls:

1. **Too complex story** - Start with 10 nodes, expand to 20
2. **Unbalanced gameplay** - Stats too easy or too hard
3. **Poor GUI readability** - Text must be clear and large
4. **Not testing different paths** - Play every branch!
5. **Ignoring game balance** - If no one can win or everyone wins easily

Testing Checklist:

- ☐ Playthrough reaching each ending
 - ☐ Test all decision paths work
 - ☐ Verify stats affect gameplay (low energy = limited choices)
 - ☐ Test inventory add/remove/use
 - ☐ Test undo functionality (5 levels)
 - ☐ Test save/load at different points
 - ☐ Verify events trigger correctly
 - ☐ Test death conditions (hunger, health)
 - ☐ Test pack recruitment (if implemented)
 - ☐ Get 3 people outside team to playtest
-

Team Collaboration Strategy

Work Distribution:

Days 2-3:

- Member 1-2: Decision tree + story writing
- Member 3-4: Stats system + inventory
- Member 5-6: Events queue + game history
- Member 7-8: GUI interface
- Report: Document game design

Days 4-5:

- All: Integration and game loop
- Pairs: Connect their modules
- Team: Playtest together
- Report: Document implementation

Day 6:

- All: Write more scenarios
- All: Playtest and balance
- All: Bug fixes
- Report: Add testing section

Creative Sessions:

- Brainstorm story scenarios together
- Vote on best ideas
- Each member writes 3-4 scenarios
- Review and give feedback

Playtesting:

- Everyone plays the game
 - Note confusing parts
 - Track where players die most
 - Adjust difficulty accordingly
-

Final Thoughts

This wolf survival game teaches you:

- How games use data structures for AI and logic
- How branching narratives work (Telltale Games style)
- Priority systems for event management
- State management and undo mechanics
- Creative problem solving

What Makes This Special:

- Most unique project among the 5
- Combines creativity with technical skills
- Fun to build and play
- Great portfolio piece
- Shows you can apply DS to any domain

Success Formula:

Good Story + Solid Data Structures + Simple GUI +
Balanced Gameplay + Team Playtesting = Awesome Game

Remember: Your game should be:

- Winnable (but challenging)
- Interesting (choices matter)
- Functional (all features work)
- Documented (explain the design)

Make a game you'd want to play!

Submission Deadline: [9th January - Friday, 11:59 PM]
